# AI ASSISTENT CODING - 14.4

**Name :-p.charankumar**
**Roll no :-2303A52138**
**Batch:-41**
**Task 1:**

Responsive Homepage Layout for a
Startup Website
Scenario
You are designing a basic homepage for a startup company that needs
to look good on desktops, tablets, and mobile devices.
Task Description
• Create a web page structure using HTML that includes:
o A header with navigation links
o A main content section
o A footer with basic information
• Use AI assistance to generate responsive CSS using media
queries.
• Ensure the layout adjusts smoothly across different screen sizes.

**Prompt:**
Create a responsive startup company homepage using HTML and CSS. The webpage should include a header with a title and navigation links (About Us, Services, Contact). Add a main section with three sections describing the startup, the services offered (like Web Development, Mobile App Development, Digital Marketing, and Consulting), and contact information. Include a footer with copyright text. Use CSS styling and media queries so that the navigation links stack vertically on smaller screens such as tablets and mobile devices. The design should be simple, clean, and responsive.

**Code:**

```
AI.html > html > head > style > nav a
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Startup Company</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
        }
        header {
            background-color: #555;
            color: #fff;
            padding: 20px;
            text-align: center;
        }
        nav a {
            color: #fff;
            margin: 0 15px;
            text-decoration: none;
        }
        main {
            padding: 20px;
        }
        footer {
            background-color: #555;
            color: #fff;
            text-align: center;
```

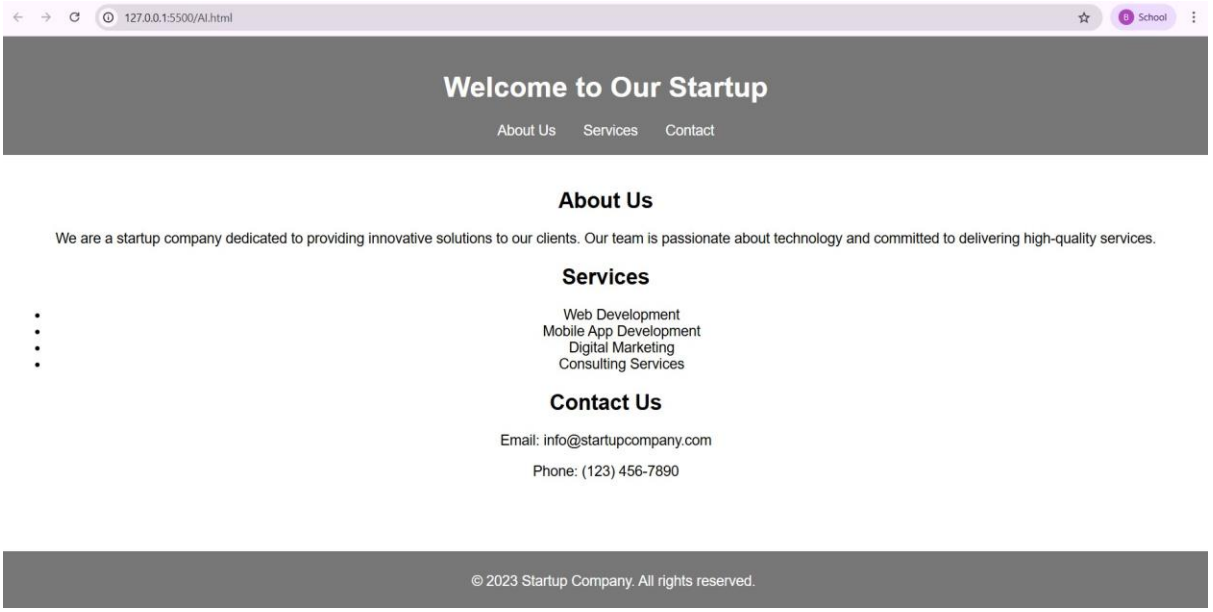
```
31              padding: 10px;
32              position: fixed;
33              width: 100%;
34              bottom: 0;
35          }
36          @media (max-width: 768px) {
37              nav a {
38                  display: block;
39                  margin: 10px 0;
40              }
41          }
42          /* Custom styles */
43          body {
44              font-size: 18px;
45              text-align: center;
46          }
47          header {
48              background-color: #777; /* Lighter background color */
49              text-align: center;
50          }
51          footer {
52              background-color: #777; /* Lighter background color */
53              text-align: center;
54          }
55      </style>
```

```
56  </head>
57  <body>
58      <header>
59          <h1>Welcome to Our Startup</h1>
60          <nav>
61              <a href="#about">About Us</a>
62              <a href="#services">Services</a>
63              <a href="#contact">Contact</a>
64          </nav>
65      </header>
66
67      <main>
68          <section id="about">
69              <h2>About Us</h2>
70              <p>We are a startup company dedicated to providing innovative soluti
71          </section>
72
73          <section id="services">
74              <h2>Services</h2>
75              <ul>
76                  <li>Web Development</li>
77                  <li>Mobile App Development</li>
78                  <li>Digital Marketing</li>
79                  <li>Consulting Services</li>
80              </ul>
81          </section>
```

```html
        <section id="contact">
            <h2>Contact Us</h2>
            <p>Email: info@startupcompany.com</p>
            <p>Phone: (123) 456-7890</p>
        </section>
    </main>

    <footer>
        <p>&copy; 2023 Startup Company. All rights reserved.</p>
    </footer>
</body>
</html>
```

**Output:**




**Justification:**
AI was used to generate responsive CSS with media queries for the startup homepage layout. This helps the website adjust properly on **desktop, tablet, and mobile screens**. It improves **user experience and accessibility** by ensuring content fits different screen sizes. Using AI also **saves development time and helps create a clean, responsive design quickly**.

## Task 2:

Interactive Call-to-Action Button
Scenario
A marketing page requires a call-to-action button that responds when users interact with it.
Task Description

• Add a button to a web page using HTML.
• Use AI to generate JavaScript that responds to the button click by:
o Displaying a message or alert
• Ask AI to add meaningful comments explaining the JavaScript Logic.

**Prompt:**
Create a simple web page using HTML and JavaScript that includes a Call-to-Action button labeled ‘Get Started’ or ‘Click Me’. When the user clicks the button, JavaScript should display an alert message thanking the user or confirming the action. Add clear comments in the JavaScript code explaining what each part of the code does. Keep the design simple and easy to understand.

**Code:**

```html
```html<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple Web Page</title>
    <style>
        /* Basic styling for the body of the page */
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
        }
        /* Styling for the Call-to-Action button */
        .cta-button {
            background-color: #4CAF50; /* Green background */
            color: white; /* White text */
            padding: 15px 32px; /* Padding around the text */
            text-align: center; /* Center the text */
            text-decoration: none; /* Remove underline from links */
            display: inline-block; /* Make the button inline-block */
            font-size: 16px; /* Set font size */
            margin: 20px; /* Margin around the button */
            cursor: pointer; /* Change cursor to pointer on hover */
        }
```
```

```
        .cta-button:hover {
            background-color: #45a049; /* Darker green on hover */
        }
    </style>
</head><body>
    <h1>Welcome to Our Website!</h1>
    <p>Click the button below to get started.</p>
    <!-- Call-to-Action button -->
    <button class="cta-button" onclick="handleButtonClick()">Get Started</button
    <script>
        // This function is called when the user clicks the Call-to-Action butto
        function handleButtonClick() {
            // Display an alert message thanking the user for clicking the butto
            alert("Thank you for getting started! We appreciate your interest.")
        }
    </script>
</body>
</html>
```

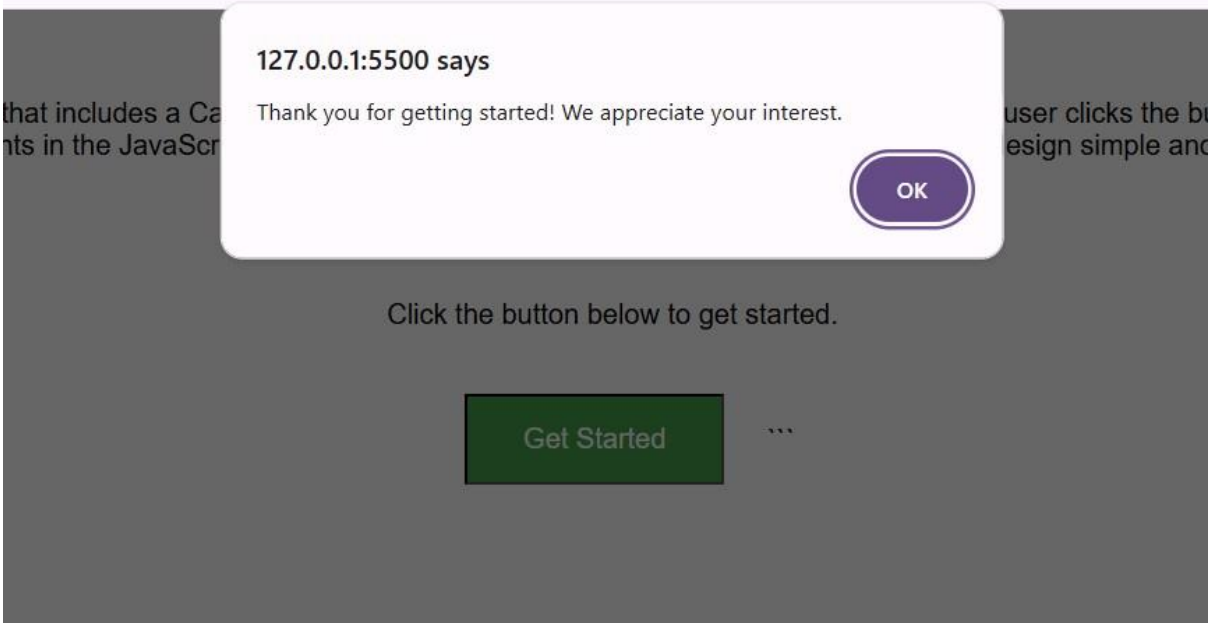
## Output:

#"Create a simple web page using HTML and JavaScript that includes a Call-to-Action button labeled 'Get Started' or 'Click Me'. When the user clicks the button, JavaScript should display an alert message thanking the user or confirming the action. Add clear comments in the JavaScript code explaining what each part of the code does. Keep the design simple and easy to understand." from pydoc import html ```html

**Welcome to Our Website!**

Click the button below to get started.

Get Started ```



**Justification:**
AI was used to generate JavaScript code that makes the call-to-action button interactive. When the user clicks the button, it displays an alert message, improving user interaction. The AI also adds comments to explain the logic, making the code easier to understand. This helps developers quickly implement interactive features on a webpage.

**Task 3:**
Contact Form with Client-Side
Validation
Scenario
You are developing a contact form for a company website to collect
user queries safely and correctly.
Task Description
• Design a form with fields for:
o Name
o Email
o Message
• Use AI to generate JavaScript validation logic that:
o Prevents empty submissions
o Checks for a valid email format
• Display clear error messages for invalid input.

**Prompt:**
Create a contact form using HTML that includes input fields for Name, Email, and Message, along with a Submit button. Generate JavaScript client-side validation that checks if all fields are filled and prevents the form from submitting if any field is empty. The script should also validate the email format to ensure it is correct. If the input is invalid, display clear error messages to guide the user to enter correct information.

**Code:**

AI3.html > html > head > style > .error

```
#Create a contact form using HTML that includes input fields for Name, Email, and Message, alo
import html

```html<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .error {
            color: red;
            margin-top: 5px;
        }
        .form-group {
            margin-bottom: 15px;
        }
    </style>
</head>
<body>
    <h1>Contact Us</h1>
    <form id="contactForm" onsubmit="return validateForm()">
        <div class="form-group">
            <label for="name">Name:</label><br>
            <input type="text" id="name" name="name"><br>
            <span id="nameError" class="error"></span>
        </div>
        <div class="form-group">
            <label for="email">Email:</label><br>
            <input type="text" id="email" name="email"><br>
            <span id="emailError" class="error"></span>
        </div>
        <div class="form-group">
            <label for="message">Message:</label><br>
            <textarea id="message" name="message"></textarea><br>
            <span id="messageError" class="error"></span>
        </div>
        <button type="submit">Submit</button>
    </form>
    <script>
        function validateForm() {
            let isValid = true;

            // Clear previous error messages
            document.getElementById('nameError').textContent = '';
            document.getElementById('emailError').textContent = '';
            document.getElementById('messageError').textContent = '';

            // Get form values
            const name = document.getElementById('name').value.trim();
            const email = document.getElementById('email').value.trim();
```
```

```
45          function validateForm() {
60                  document.getElementById('nameError').textContent = 'Please enter your name.';
61                  isValid = false;
62              }
63
64              // Validate Email
65              const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
66              if (email === '') {
67                  document.getElementById('emailError').textContent = 'Please enter your email.';
68                  isValid = false;
69              } else if (!emailPattern.test(email)) {
70                  document.getElementById('emailError').textContent = 'Please enter a valid email addr
71                  isValid = false;
72              }
73
74              // Validate Message
75              if (message === '') {
76                  document.getElementById('messageError').textContent = 'Please enter your message.';
77                  isValid = false;
78              }
79
80              return isValid; // Prevent form submission if any field is invalid
81          }
82      </script>
83  </body>
84  </html>
85  ```
```

**Output:**

# Contact Us

Name:

Vaishnavi Baisa

Email:

vaishnavibaisa05@gmail.cc

Message:

Hellooooo

Submit

```
```

# Contact Us

Name:
VAISHNAVI

Email:
vaishnavibaisa05@.com
Please enter a valid email address.

Message:
HIIIII

Submit

```

**Justification:**
AI was used to generate client-side validation using JavaScript for the contact form. It ensures that users cannot submit empty fields and that the email address follows a valid format. Error messages are displayed to guide users in correcting their input. This improves data accuracy and user experience on the website.
```

**Task 4:**
Dynamic List Management for a
Product Page
Scenario
A product listing page needs to add or remove items dynamically
without refreshing the page.
Task Description
• Create a simple HTML list to display items.
• Use AI to generate JavaScript that:
o Adds a new item when an “Add” button is clicked
o Removes the last item when a “Remove” button is clicked
• Ensure the DOM updates dynamically.

**Prompt:**
Create a simple product list webpage using HTML and JavaScript. The page should display a list of items and include two buttons: ‘Add Item’ and ‘Remove Item’. When the Add Item button is clicked, JavaScript should dynamically add a new item to the list. When the Remove Item button is clicked, it should remove the last item from the list. Ensure the DOM updates dynamically without refreshing the page and include comments explaining the JavaScript code.

## Code:

AI4.html > ...

```html
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product List</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        #product-list {
            list-style-type: none;
            padding: 0;
        }
        #product-list li {
            padding: 10px;
            border: 1px solid #ccc;
            margin-bottom: 5px;
        }
        button {
            margin-right: 10px;
            padding: 10px 20px;
            font-size: 16px;
        }
    </style>
</head>
<body>
```

```
    <h1>Product List</h1>
    <ul id="product-list">
        <!-- Product items will be added here dynamically -->
    </ul>
    <button id="add-item">Add Item</button>
    <button id="remove-item">Remove Item</button>

    <script>
        // Get references to the buttons and the product list
        const addItemButton = document.getElementById('add-item');
        const removeItemButton = document.getElementById('remove-item');
        const productList = document.getElementById('product-list');

        // Initialize a counter to keep track of the number of items
        let itemCount = 0;

        // Event listener for the 'Add Item' button
        addItemButton.addEventListener('click', () => {
            itemCount++; // Increment the item count
            const newItem = document.createElement('li'); // Create a new list item element
            newItem.textContent = `Product ${itemCount}`; // Set the text content of the new item
            productList.appendChild(newItem); // Add the new item to the product list
        });

```
```

```
        // Event listener for the 'Remove Item' button
        removeItemButton.addEventListener('click', () => {
            if (productList.lastChild) { // Check if there are any items in the list
                productList.removeChild(productList.lastChild); // Remove the last item from the list
                itemCount--; // Decrement the item count
            }
        });
    </script>
</body>
</html>
```

**Output:**

# Product List

Product 0

Product 1

Product 2

Product 3

Add Item  Remove Item  ```

After removing Items

# Product List

Product 0

Product 1

Add Item  Remove Item  ```

**Justification:**

AI was used to generate JavaScript that allows dynamic addition and removal of items in the product list. The DOM updates instantly without refreshing the page, improving user interaction and responsiveness. This makes the product page more interactive and efficient. AI also helps generate clean and understandable code quickly.

**Task 5:**
Modal Popup for User Notifications
Scenario
You are building a user notification popup for announcements or alerts on a website.
Task Description
• Use AI to help generate:
o HTML structure for a modal popup
o CSS styling with a semi-transparent background overlay
o JavaScript to open and close the modal
• Include multiple ways to close the modal (button or overlay click).

**Prompt:**
Create a modal popup notification for a website using HTML, CSS, and JavaScript. The HTML should include the structure of the modal popup with a message and a close button. Use CSS to style the modal with a centered popup box and a semi-transparent background overlay. Generate JavaScript code to open and close the modal popup. The modal should close when the user clicks the close button or the background overlay. Add comments in the JavaScript to explain the functionality.

**Code:**

AI5.html > html > head > style > .modal-overlay

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modal Popup Example</title>
    <style>
        /* Style for the modal background overlay */
        .modal-overlay {
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5); /* Semi-transparent background */
            display: none; /* Hidden by default */
            justify-content: center; /* Center the modal horizontally */
            align-items: center; /* Center the modal vertically */
        }

        /* Style for the modal popup box */
        .modal {
            background-color: white;
            padding: 20px;
            border-radius: 5px;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
            text-align: center;
        }
```

```
        /* Style for the close button */
        .close-button {
            margin-top: 15px;
            padding: 10px 20px;
            background-color: #007BFF;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <!-- Button to open the modal -->
    <button id="openModal">Open Modal</button>

    <!-- Modal structure -->
    <div class="modal-overlay" id="modalOverlay">
        <div class="modal">
            <p>This is a modal popup notification!</p>
            <button class="close-button" id="closeModal">Close</button>
        </div>
    </div>

```

```
    <script>
        // Get references to the modal elements
        const openModalButton = document.getElementById('openModal');
        const closeModalButton = document.getElementById('closeModal');
        const modalOverlay = document.getElementById('modalOverlay');

        // Function to open the modal
        function openModal() {
            modalOverlay.style.display = 'flex'; // Show the modal by setting display to flex
        }

        // Function to close the modal
        function closeModal() {
            modalOverlay.style.display = 'none'; // Hide the modal by setting display to none
        }

        // Event listener for the open modal button
        openModalButton.addEventListener('click', openModal);

        // Event listener for the close modal button
        closeModalButton.addEventListener('click', closeModal);

        // Event listener for clicking outside the modal (on the overlay)
        modalOverlay.addEventListener('click', function(event) {
            if (event.target === modalOverlay) { // Check if the click is on the overlay, not the
                closeModal(); // Close the modal if the overlay is clicked
            }
```

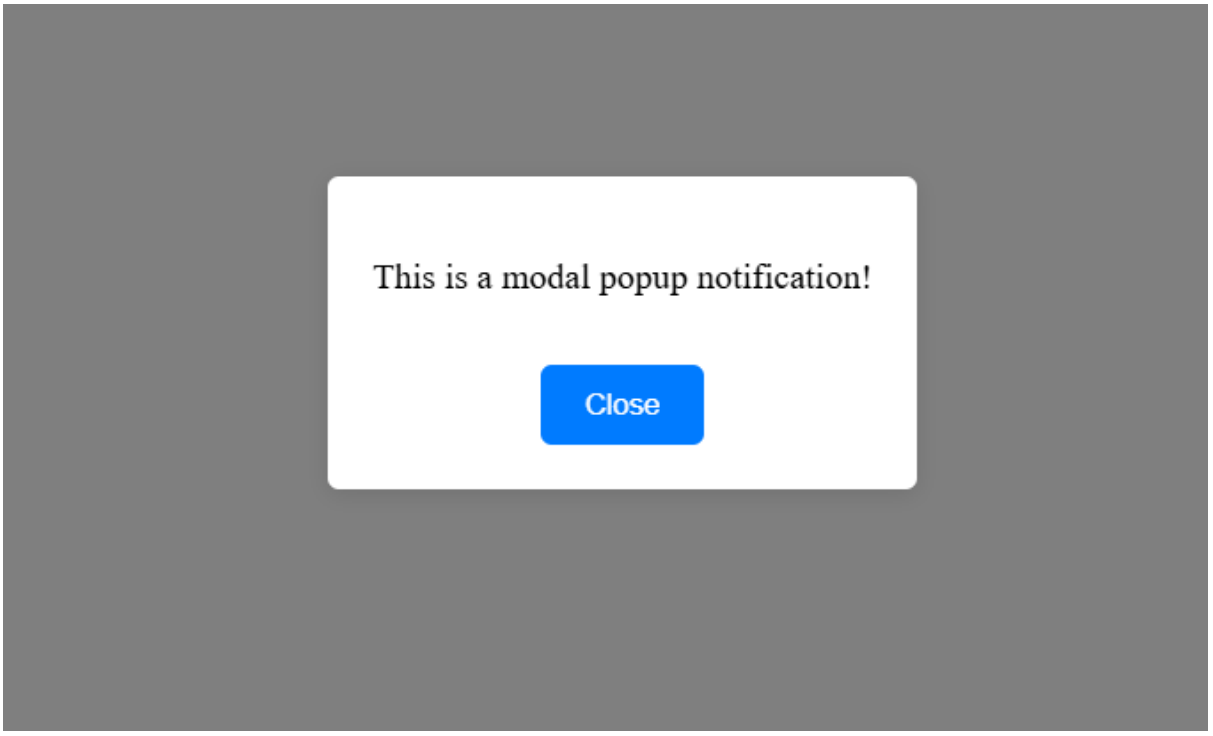
```
            }
        });
    </script>
</body>
</html>
```

## Output:

```html
```

Open Modal

```
```




**Justification:**
AI was used to generate the **modal popup structure, styling, and JavaScript functionality**. The popup displays notifications without leaving the current page. Multiple closing options, such as **clicking the close button or overlay**, improve usability. This enhances **user experience and makes website notifications more interactive and accessible**.